CODELENS KNOWLEDGE BASE - DATABASE CONNECTIONS & CONTEXT MANAGERS (PAGE 1)

==========

Module: Database Connection Pooling and Resource Safety Guide

1. DATABASE CONNECTION LEAKS IN BACKEND SERVICES

A common critical issue in backend services is failing to close database connections.
When a connection is opened via db.connect() without a context manager, any runtime
exception in subsequent queries causes the connection to remain open indefinitely.

Symptom: OperationalError: Too many connections / connection pool exhaustion.

Problematic Pattern:

```
conn = db_pool.get_connection()
cursor = conn.cursor()
cursor.execute('SELECT * FROM users WHERE active = 1')
records = cursor.fetchall()
# If an error happens above, conn.close() is never reached!
conn.close()
```

2. SAFE RESOURCE RECOVERY USING CONTEXT MANAGERS

In Python, always manage connections using the 'with' statement or try-finally blocks.
The context manager guarantees that the __exit__ method executes conn.close() or
returns the connection back to the connection pool, even if an exception is raised.

CODELENS KNOWLEDGE BASE - DATABASE CONNECTIONS & CONTEXT MANAGERS (PAGE 2)

==========

Module: Recommended Database Cleanup Idioms and Best Practices

3. THE RECOMMENDED CONNECTION CONTEXT MANAGER PATTERN

Standard production idiom for leak-proof database transactions:

```
    with db_pool.get_connection() as conn:
        with conn.cursor() as cursor:
            cursor.execute('SELECT * FROM users WHERE active = 1')
            records = cursor.fetchall()
```

4. EXPLICIT CLEANUP WITH TRY-FINALLY BLOCKS

If a context manager is not supported by the driver, use try-finally:

```
    conn = db_pool.get_connection()
    try:
        cursor = conn.cursor()
        cursor.execute(query)
        return cursor.fetchall()
    finally:
        conn.close()  # Guaranteed execution prevents connection leaks
```

5. RULE FOR CODE REVIEWS: Never open raw I/O or network connections without 'with'.